

NS01

在軟體工程與資訊系統開發中，落實 Jakob Nielsen 的 **Visibility of System Status**（系統狀態能見度），核心在於消除使用者的「不確定感」。

要讓 AI 在開發過程中產出符合此原則的結果，您的 Prompt 需要從**狀態反饋**、**等待機制**、**導航定位**三個維度切入。以下是針對不同階段建議的 Prompt 策略：

1. 系統架構與狀態設計 (System Logic)

在設計後端邏輯或狀態機時，Prompt 應要求系統明確定義「過渡狀態」。

- **建議 Prompt：**

「在設計 [某功能，如：檔案上傳/訂單處理] 的邏輯時，請列出所有可能的**中間狀態**（如：處理中、校驗中、同步中）。請確保 API 回傳值包含進度百分比或預估剩餘時間，並設計當系統延遲超過 2 秒時的自動通知機制。」

2. UI/UX 視覺反饋 (Visual Feedback)

針對前端元件，Prompt 應著重於「視覺化」目前的進度，避免畫面看起來像「當機」。

- **建議 Prompt：**

「針對 [某操作介面]，請設計符合 Nielsen 第一條原則的 UI 組件。當使用者點擊按鈕後，請提供：

- 立即性反饋**：按鈕轉為 Loading 狀態或顯示 Spinner。
- 進度指示**：如果是耗時任務，請加入 Progress Bar 並顯示當前步驟（例如：Step 2/4）。
- 完成提示**：任務結束後，需有明確的 Success/Error Toast 訊息。」

3. 導航與位置感知 (Navigation & Orientation)

系統狀態也包含「我現在在哪裡」，這對複雜的資訊系統尤為重要。

- **建議 Prompt：**

「在設計系統導航欄位時，請確保提供清晰的 **Breadcrumbs (麵包屑)** 與 **Active State (當前選單高亮)**。使用者必須能隨時透過視覺特徵判斷目前在系統的哪個層級，並在切換頁面時提供骨架屏 (Skeleton Screens) 以表示資料正在載入。」

實戰範例：整合型 Prompt

如果您想讓 AI 直接產出程式碼或規格書，可以嘗試這段具備「工程師思維」的指令：

Prompt:

「我正在開發一個 [系統名稱]，請針對 [特定功能] 撰寫實作建議。」

要求： 嚴格遵循 Nielsen 的 **Visibility of System Status** 原則：

1. 請列出此功能所有可能的系統狀態 (Idle, Busy, Success, Failed)。
2. 描述 UI 如何在 0.1 秒內給予使用者互動反饋。
3. 若處理時間超過 1 秒，請提供進度條或提示語的設計邏輯。
4. 請提供前端 Vue/React (可自行替換) 的狀態變數建議範例。」

專家小提醒

在與 AI 協作時，您可以特別強調**「非同步處理 (Asynchronous)」**的狀態顯示。許多開發者常忘了在資料尚未從資料庫回傳前，先給使用者一個「我正在努力中」的信號，這正是 Visibility of System Status 最常被忽略的地方。

您目前開發的這個資訊系統，主要是 Web 端還是行動裝置 App 呢？針對不同平台，狀態顯示的慣例（如下拉重新整理或骨架屏）會略有不同。